

lesson1 guide

P1. Рисуем первый игровой мир

Окно, координаты, фигуры, картинки и первые кадры.

Что получится

Сегодня ты создаёшь первый игровой мир: окно, фон, несколько объектов, картинку героя или предмета, название игры и короткую покадровую анимацию. Обязательная часть у всех похожая, а личные улучшения можно выбрать в `achievements.md`.

1. Установка на Windows

Python

1. Открой <https://www.python.org/downloads/>
2. Скачай Python 3.12 или новее.
3. На первом экране установки поставь галочку **Add python.exe to PATH**.
4. Нажми **Install Now**.
5. Открой PowerShell и проверь:

```
python --version
```

Если появилась версия Python, установка сработала.

VS Code

1. Открой <https://code.visualstudio.com/>
2. Установи VS Code.
3. Открой вкладку расширений.
4. Установи расширения:
 - Python;
 - Pylance;
 - Marp for VS Code.

Проект

1. Открой папку проекта в VS Code: **File → Open Folder**.
2. Если VS Code спросит, доверять ли папке, выбери **Trust**.
3. Открой любой файл Python, например `lessons/lesson1/samples/01_first_scene.py`.
4. В правом нижнем углу VS Code выбери интерпретатор Python.
5. Если окружение ещё не создано, выбери создание окружения через **uv**. VS Code сам прочитает `pyproject.toml` и `uv.lock` и подготовит `.venv`.
6. Дождись, пока VS Code закончит установку зависимостей.

Вручную ставить `pyglet` через `pip` не нужно. Зависимости проекта уже описаны в файлах проекта, а VS Code и `uv` создают окружение автоматически.

Если преподаватель дал готовую папку с уже созданным окружением, просто выбери интерпретатор из `.venv`.

2. Коротко для macOS и Linux

1. Установи Python 3.12 или новее.
2. Установи VS Code и расширения Python, Pylance, Marp for VS Code.
3. Открой папку проекта.
4. Открой любой файл Python.
5. Через выбор интерпретатора в VS Code создай или выбери окружение `.venv`.

6. Если VS Code предложит использовать `uv`, соглашайся: зависимости подтянутся из `uv.lock`.

Если VS Code не видит Python или окружение, спроси преподавателя: на разных системах путь к Python может отличаться.

3. Папка команды

Обычно команда состоит из двух учеников. Перед первой программой создайте отдельную папку внутри `team/`.

Структура:

```
team/  
  <team_name>/  
    __init__.py  
    README.md  
    <project1_name>.py  
    ... .py
```

Пример:

```
team/  
  space_cats/  
    __init__.py  
    README.md  
    first_scene.py  
    moon_animation.py
```

Что написать в `README.md`:

- название команды;
- имена участников;
- короткое описание команды;
- позже — описание проектов.

Правило проекта: весь код, картинки, звуки и другие файлы, которые создаёт команда, лежат внутри папки команды. Если файл нужен только вашей команде, он не должен лежать в общей папке урока.

Общие примеры остаются в `lessons/lesson1/samples/`. Их можно читать и запускать, но свой проект сохраняйте в `team/<team_name>/`.

4. Первый запуск

Открой файл:

```
lessons/lesson1/samples/01_first_scene.py
```

Запусти его через VS Code:

1. Нажми кнопку запуска Python в правом верхнем углу редактора.
2. Если VS Code спросит, чем запускать файл, выбери Python из `.venv`.
3. Дождись появления окна игры.

Что должно получиться: откроется окно с первой игровой сценой. Окно останется открытым, пока ты сам его не закроешь.

Если окно появилось, но кажется пустым, не закрывай сразу: иногда окно открывается за VS Code.

5. Как работать с кодом

- Цветной текст в редакторе называется подсветкой синтаксиса. Она помогает различать числа, строки, имена и

- команды.
- Автодополнение подсказывает имена команд. Начни писать `game.` и посмотри список.
 - После изменения файла нажми `Ctrl+S`.
 - Чтобы увидеть новый результат, закрой окно программы и запусти файл ещё раз.
 - Меняй по одному месту за раз. Так легче понять, что именно повлияло на картинку.
 - В примерах этого урока нет новых конструкций для повторов и своих команд. Мы пока пишем команды прямо по порядку.

6. Что можно менять безопасно

В примерах сначала меняй:

- числа координат `x` и `y`;
- размеры фигур;
- названия цветов;
- текст заголовка;
- количество объектов;
- переменные вроде `hero_x` или `ground_y`.

Не меняй сразу названия команд `draw_circle`, `clear_canvas`, `show_canvas`: если в имени будет ошибка, Python не поймёт команду.

7. Что важно понять про графику

- Окно игры — это сцена, где появляется игровой мир.
- Экран можно представить как лист в клетку.
- Координаты говорят компьютеру, где рисовать объект.
- `x` идёт вправо, `y` идёт вниз.
- Компьютер выполняет команды строго по порядку.
- Если фон нарисован после героя, фон закроет героя.
- Чтобы картинка ожила, мы показываем несколько кадров подряд.
- Один кадр: очистили экран, нарисовали новую картинку, показали, чуть-чуть подождали.
- На этом уроке кадры записаны руками. Настоящие повторения и игровой цикл будут позже.

8. Карта понятий

Понятие	Простое объяснение
Пиксель	Маленькая точка экрана
Растр	Картинка из пикселей
Цвет RGB	Смесь красного, зелёного и синего
Файл для игры	Картинка, звук или другой файл, который использует игра
Координаты	Адрес места на экране
Вектор	Сдвиг: куда и на сколько
Кадр	Одна картинка из будущего движения
sleep	Пауза, чтобы человек успел увидеть кадр

9. Частые ошибки Python

SyntaxError

Python сообщает: «Я не понял запись». Часто не хватает скобки, кавычки или запятой.

Что делать: посмотри на строку из сообщения и строку прямо перед ней.

NameError

Python сообщает: «Такого имени я не знаю». Например, было `hero_x`, а написали `hero_x` или `hero`.

Что делать: сравни имя в ошибке с тем местом, где переменная создаётся.

IndentationError

Python сообщает: «С отступами что-то не так». Отступы показывают, какие строки относятся к одному блоку.

Что делать: проверь пробелы в начале строк. В одном блоке отступ должен быть одинаковым.

TypeError

Python сообщает: «Команда получила не то, что ожидала». Например, для круга забыли указать радиус.

Что делать: проверь, сколько значений передано в команду.

ValueError

Python сообщает: «Я понял, что это значение, но оно не подходит». Например, цвет `skyblue` не входит в список известных имён.

Что делать: используй знакомый цвет или HEX-рецепт вроде `#44aaff`.

Файл не найден

Если картинка не загрузилась, путь к файлу может быть неправильным.

Что делать: проверь папку, расширение файла и пробелы в имени.

Регистр букв

`Hero.png` и `hero.png` могут быть разными именами.

Что делать: копируй имя файла точно.

10. Задания-поломки

Файлы в `lessons/lesson1/puzzles/` специально ведут себя неправильно. Это не примеры для копирования, а маленькие площадки для поиска и исправления ошибок.

В этих заданиях ошибки не только в синтаксисе. Чаще нужно понять, что происходит со сценой: что было нарисовано раньше, что закрыло героя, почему кадр промелькнул, почему объект оказался за экраном или почему не нашлась картинка.

Правило:

1. Запусти задание.
2. Посмотри на ошибку или странный результат.
3. Найди место, где программа сбилась.
4. Исправь одну вещь.
5. Запусти снова.

Не пытайся чинить всё сразу.

11. Финальная сборка проекта

1. Создай папку команды в `team/`.
2. Возьми за основу идею из `lessons/lesson1/samples/01_first_scene.py` и перенеси её в свой файл внутри папки команды.
3. Измени фон и цвета.

4. Переставь объекты по координатам.
5. Добавь картинку героя или предмета.
6. Добавь название игры.
7. Открой `lessons/lesson1/samples/05_manual_animation.py`.
8. Сделай 3 кадра небольшого движения в файле своей команды.
9. Открой `lessons/lesson1/samples/06_repeated_object.py` и посмотри, почему повторять сложный объект руками неудобно.
10. Открой `lessons/lesson1/samples/07_example_use_function.py` и сравни: повтор вынесен в одну свою команду.
11. Если добавляешь свои картинки или другие файлы, положи их в папку команды.
12. Отметь достижения в `achivements.md`.